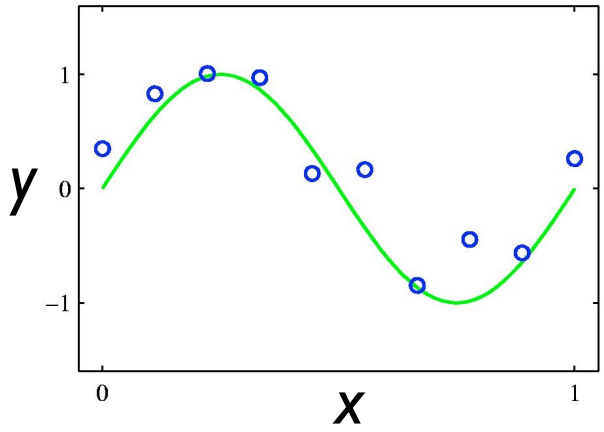
Aprendizaje Supervisado: Conceptos básicos.

Antonio M. López Rodríguez

Regresión

- Tenemos un conjunto de datos (puntos azules) y buscamos una función que modele el proceso que los genera para predecir su respuesta ante nuevas entradas (supongamos que el proceso viene descrito por la curva verde, aunque obviamente nosotros no la conoceríamos).



- Aprovecharemos este problema para introducir los conceptos fundamentales de Machine Learning, extensibles a otros muchos problemas.

Gráficas tomadas de [Bishop]

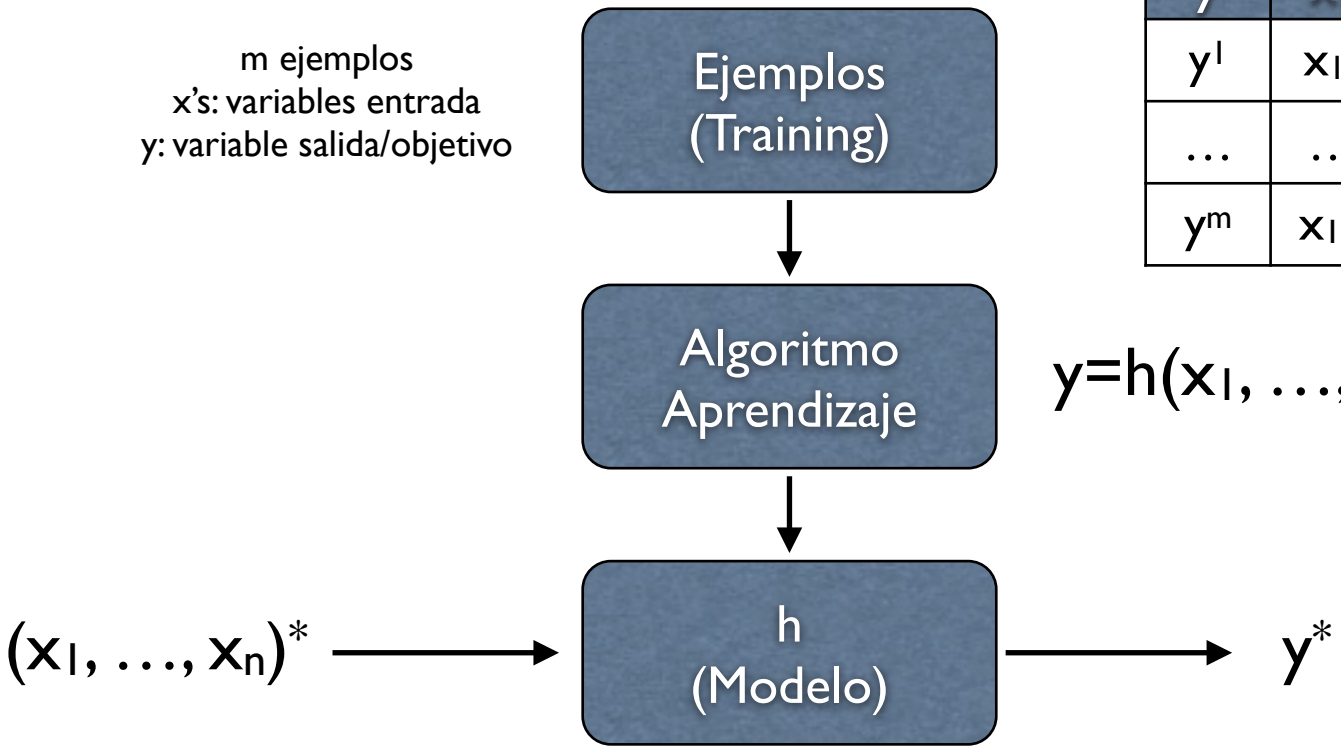
Metodología General Aprendizaje Automático

- Trabajaremos:
  - **Planteamiento del problema**
  - Definición del modelo (hipótesis).
  - Definición de la Función de coste.
  - Minimización de la función de coste.

Planteamiento del Problema

m ejemplos
x's: variables entrada
y: variable salida/objetivo

y	x_1	…	x_n
y^1	$x_1{}^1$		$x_n{}^1$
…	…	…	…
y^m	$x_1{}^m$		$x_n{}^m$



Metodología General Aprendizaje Automático

- Trabajaremos:
  - Planteamiento del problema
  - **Definición del modelo (hipótesis).**
  - Definición de la Función de coste.
  - Minimización de la función de coste.

Definición del modelo (hipótesis)

- De momento nos limitaremos a considerar modelos lineales, posiblemente en un espacio de entrada extendido [1].
- Por ejemplo, vamos a suponer que *h* es una función polinómica en x:

$$h(x,a) = h_a(x) = a_0 + a_1 x + \cdots + a_n x^n = \sum_{j=0}^{n} a_j x^j$$
$$a = [a_0 \; a_1 \ldots a_n]$$

$x_n = x^n$ .....▶

y	x_0	x_1	…	x_n
y^l	l	$x_1{}^l$		$x_n{}^l$
…	…	…	…	…
y^m	l	$x_1{}^m$		$x_n{}^m$

- Planteamiento: perseguimos ajustar *a* tal que que $h_a(x) \approx y$ en los ejemplos disponibles.
- De momento supondremos un polinomio de grado fijo (n).

¿Qué valores escogemos para $a=(a_0, a_1, \ldots, a_n)$? Necesitamos una medida de la similitud entre $h_a(x)$ e *y* para los ejemplos disponibles: **Función de coste**.

[1] El problema original era claramente de una naturaleza no lineal. Redefiniendo el espacio de entrada mediante una expansión polinómica lo convertimos en un problema de carácter lineal.

Metodología General Aprendizaje Automático

- Trabajaremos:
  - Planteamiento del problema
  - Definición del modelo (hipótesis).
  - **Definición de la Función de coste.**
  - Minimización de la función de coste.

Definición de la Función de Coste

- Podemos usar el error cuadrático medio para medir la diferencia entre $h_a(x)$ e y para los ejemplos disponibles.

$$h(x,a) = h_a(x) = a_0 + a_1 x + \cdots + a_n x^n = \sum_{j=0}^{n} a_j x^j$$
$$a = [a_0 \; a_1 \ldots a_n]$$

$$J(a) = \frac{1}{2m} \sum_{i=1}^{m} (h_a(x^{(i)}) - y^{(i)})^2$$

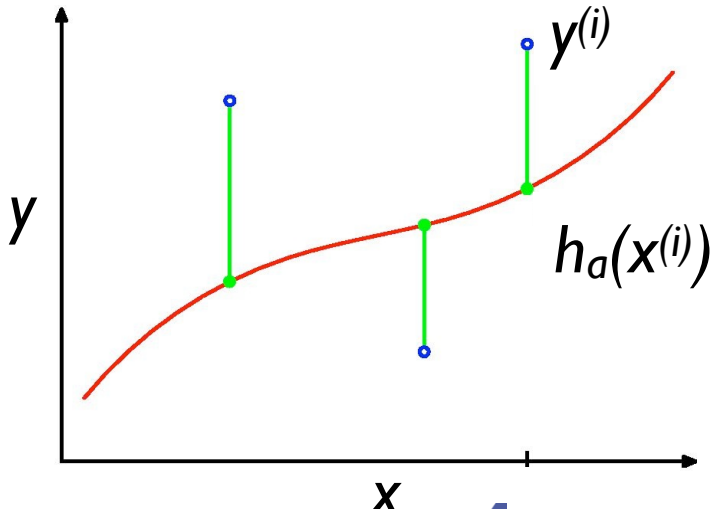
- La función de coste se define a partir de una función de pérdida. En este caso:

$$\mathcal{L}(h_a(x^{(i)}), y^{(i)})) = (h_a(x^{(i)}) - y^{(i)}))^2$$

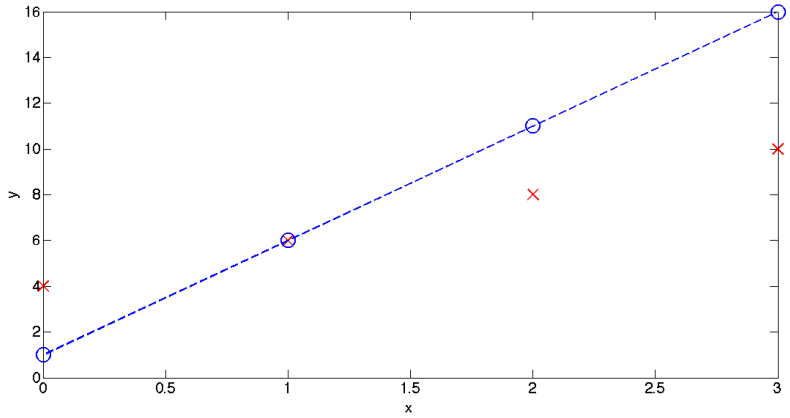
  - El objetivo será buscar el vector ***a*** que haga mínima esa función de coste con los ejemplos disponibles.

$x_n = x^n$

y	x_1	…	x_n
y^1	$x_1{}^1$		$x_n{}^1$
…	…	…	…
y^m	$x_1{}^m$		$x_n{}^m$



Función de Coste: ejemplo

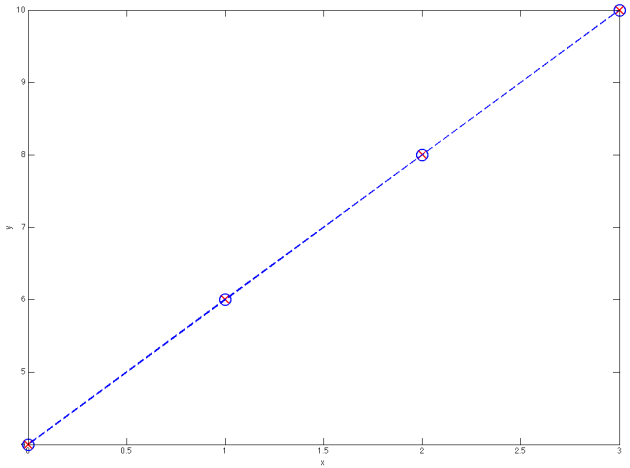


y	x
4	0
6	1
8	2
10	3

$h_a(x)=a_0+a_1x$ $\quad a_0=1$, $a_1=5$

$J([a_0, a_1]) = ?$

Función de Coste: ejemplo

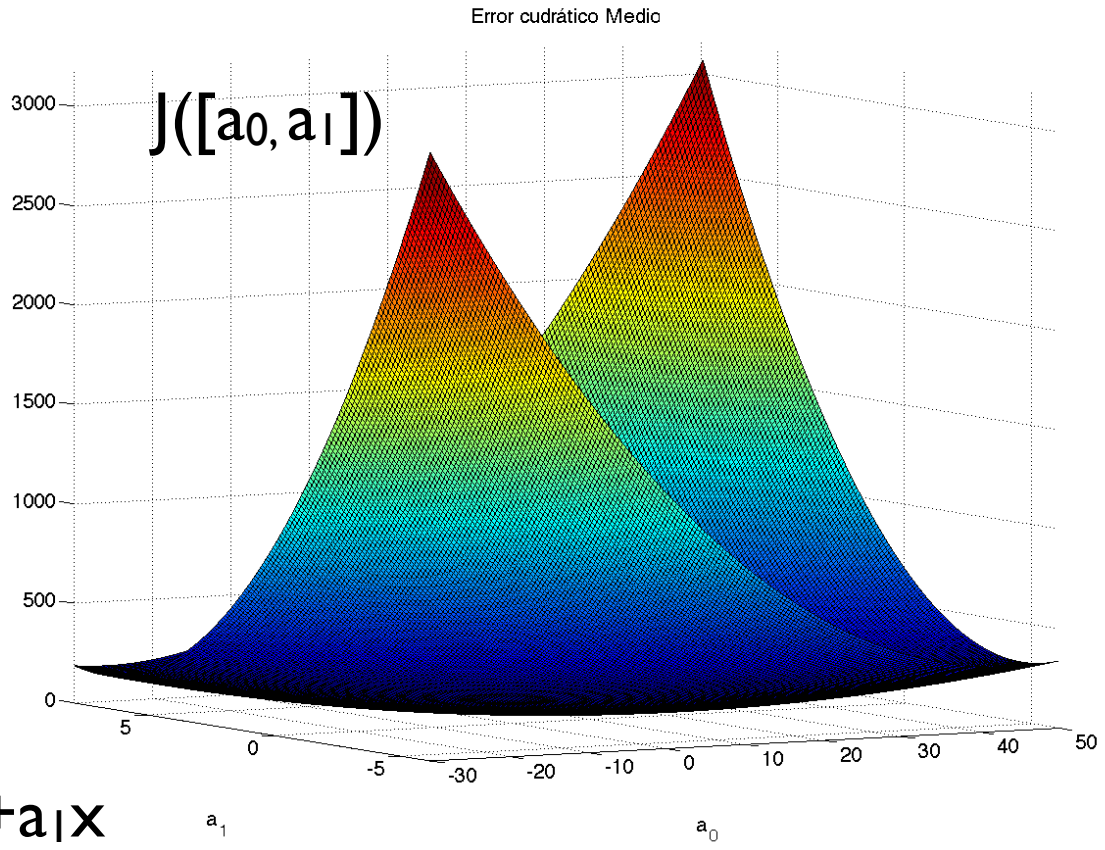


y	x
4	0
6	1
8	2
10	3

$h_a(x)=a_0+a_1x$ $\quad a_0=4$ $\quad a_1=2$

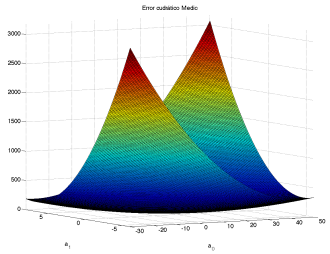
$J([a_0, a_1]) = ?$

Función de Coste - 2 dimensiones

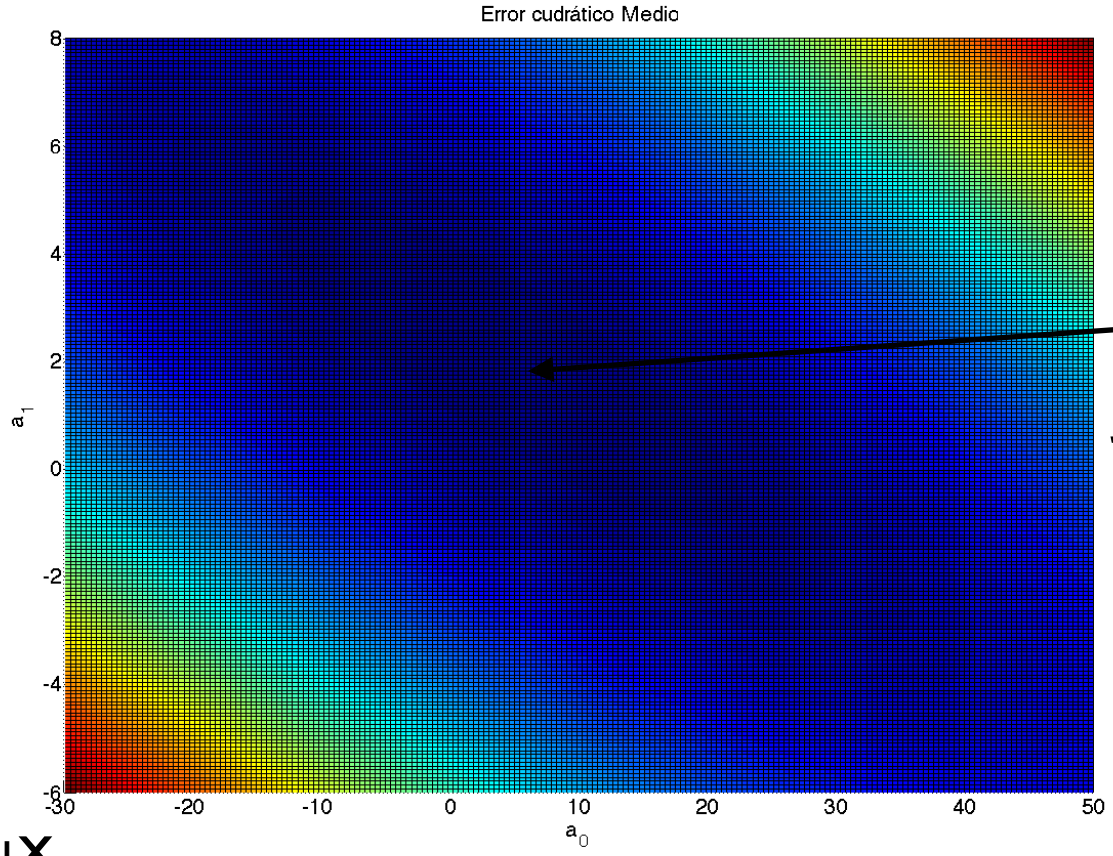


$h_a(x)=a_0+a_1x$

Función de Coste - 2 dimensiones



$J([a_0, a_1])$



$a_0=4$

$a_1=2$

$J([a_0, a_1])=0$

$h_a(x)=a_0+a_1x$

Metodología General Aprendizaje Automático

- Trabajaremos:
  - Planteamiento del problema
  - Definición del modelo (hipótesis).
  - Definición de la Función de coste.
  - **Minimización de la función de coste.**

Minimización por Descenso de gradiente

– Método iterativo de minimización muy empleado.

– El procedimiento es:

Inicializar $a=[a_0, a1, \ldots, a_n]$ (generalmente al azar)

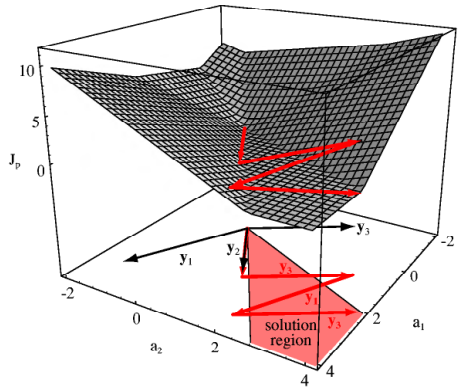
k=1.

Hacer

k=k+1;

$a=a-\alpha(k)\nabla J(a)$

hasta $|\alpha(k)\nabla J(a) < \theta|$ o $(k>k_{max})$



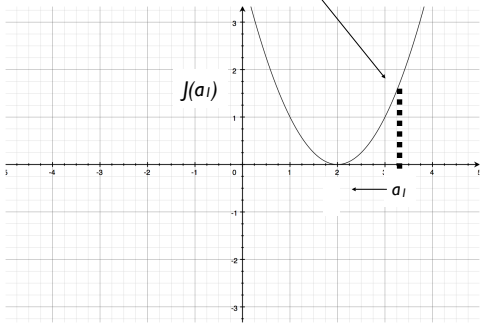
$$\nabla J(a_k) = \begin{bmatrix} \frac{\partial}{\partial a_1} J(a) \big]_{a=a_k} \\ \vdots \\ \frac{\partial}{\partial a_n} J(a) \big]_{a=a_k} \end{bmatrix}$$

α puede ser constante o variable

Descenso de Gradiente: Intuición

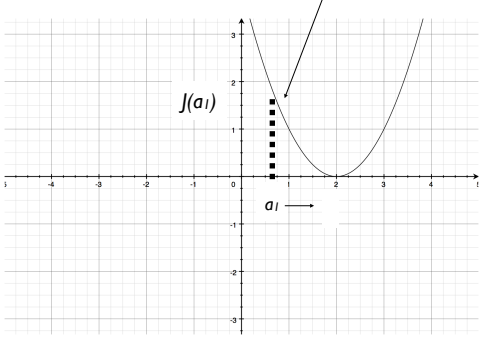
$$a_1 = a1 - \alpha \frac{\partial}{\partial a_1} J(a_1)$$

pendiente positiva



$a_1 = a1 - numero\ positivo$

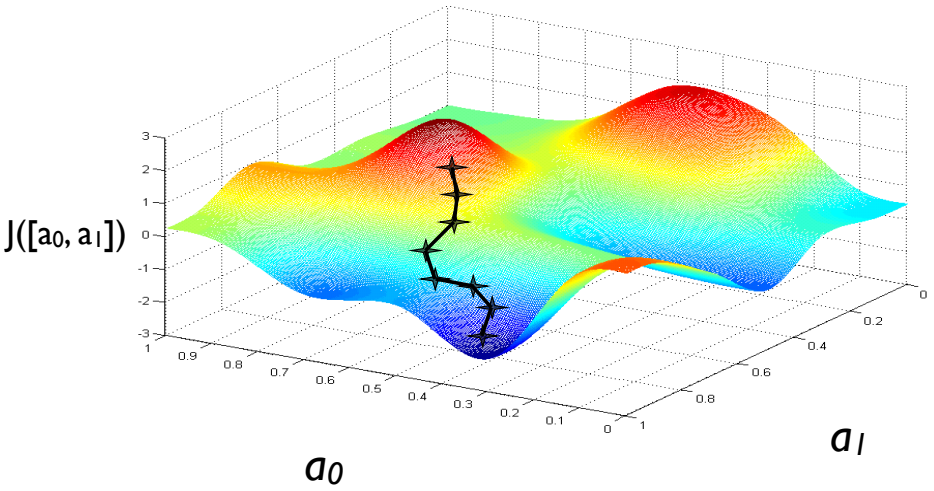
pendiente negativa

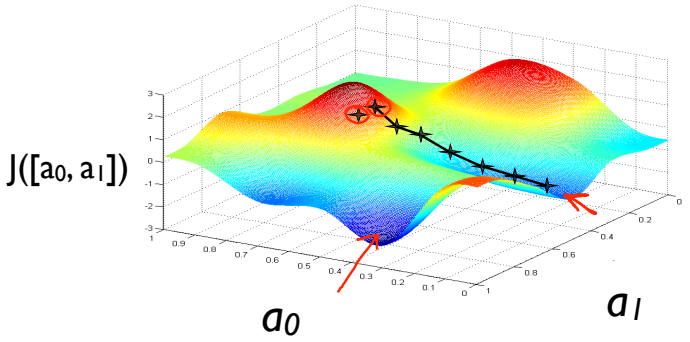


$a_1 = a1 - numero\ negativo$

Descenso de Gradiente

- Se basa únicamente en consideraciones de la primera derivada, luego solo tiene en cuenta las FONC, y por tanto puede converger a mínimos locales y puntos de silla.





Descenso de Gradiente: Algoritmo

- Supongamos $J(a_0, a_1)$: optimización bidimensional.
- Actualización:

$$a_{0_{k+1}} = a_{0_k} - \alpha \frac{\partial}{\partial a_0} J(a)]_{a=(a_{0_k}, a_{1_k})}$$

$$a_{1_{k+1}} = a_{1_k} - \alpha \frac{\partial}{\partial a_1} J(a)]_{a=(a_{0_k}, a_{1_k})}$$

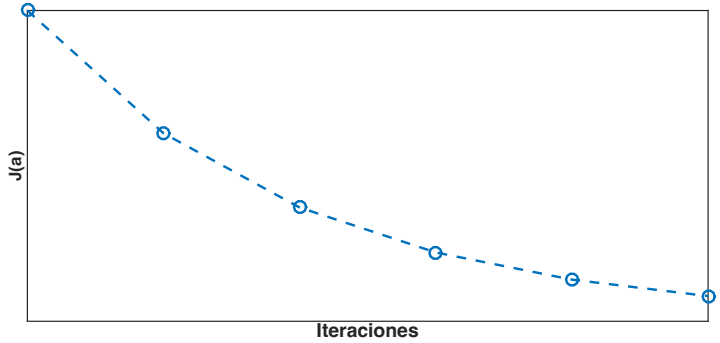
- Ojo a la programación: ¿estaría bien el siguiente código para actualizar a en una determinada iteración?

```
a0=a0-alfa*J_0(a0, a1);
a1 = a1-alfa*J_1(a0, a1)
```

(suponemos que J_0 evalúa el parcial de J respecto a a0; idem para J_1)

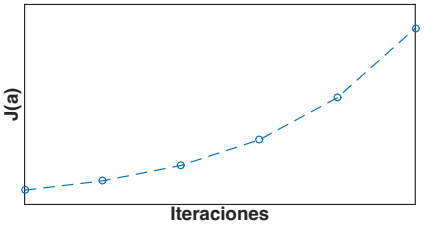
Descenso de Gradiente: Parámetro de descenso (α)

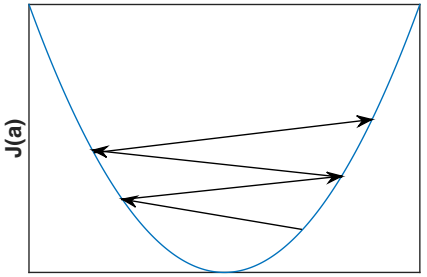
- Para un α adecuado se debería apreciar un descenso en J(a) en cada iteración con tendencia a cero:



- No obstante si es muy pequeño la convergencia será muy lenta.

- En otros casos fallaría, debido posiblemente a un alfa muy elevado:





- Podemos partir de un α e ir incrementándolo/decrementándolo geométricamente mediante un factor (e.j.:3): …., 0.001, 0.003, 0.01, 0.03, 0.1, 0.3 1, …
- No es estrictamente necesario variar α para asegurar la convergencia: a medida que nos acerquemos al valle el avance va a ser menor por efecto de menor gradiente.

Minimización de la Función de Coste
Ecuación Normal para Regresión Lineal

Datos

Rendimiento (y)	concentración (x_1)	temperatura (x_2)
81	1	150
89	1	180
83	2	150
91	2	180
79	1	150
87	1	180
84	2	150
90	2	180

El rendimiento de una reacción química depende de la temperatura de operación y de la concentración inicial del reactivo

(fuente: wikipedia)

Modelo: $h_a(x)=a_0+a_1x_1+\ldots+a_nx_n$

Independiente (x_0)	concentración (x_1)	temperatura (x_2)
1	1	150
1	1	180
1	2	150
1	2	180
1	1	150
1	1	180
1	2	150
1	2	180

Matriz características aumentada (X)

Rendimiento (y)
81
89
83
91
79
87
84
90

Matriz salida (Y)

$$a=(X^TX)^{-1}X^Ty$$

En general es mejor usar la pseudoinversa (Moore-Penrose) pues la inversión de X^T*X es frecuentemente problemática: a=pinv(X)*y

(ver https://en.wikipedia.org/wiki/Normal_equations#Derivation_of_the_normal_equations)

Minimización de la Función de Coste
¿Descenso gradiente o ecuación normal?

- Descenso gradiente
  - Iterativo
  - Conveniente “normalizar”
  - Necesario ajustar alfa
  - Funciona bien para cualquier número de características
- Ecuación normal
  - Directo
  - Innecesario normalizar
  - Ningún parámetro de diseño
  - La inversión de la matriz X^TX puede ser demasiada costosa para un número de características elevado

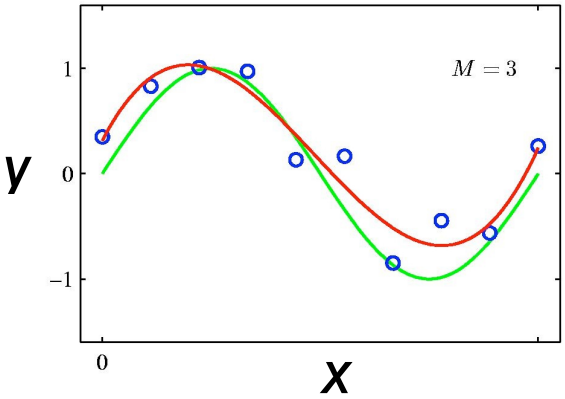
Entrenamiento, Validación, Test

- En un proceso de aprendizaje automático, se suelen abordar tres procesos claramente diferenciados:
  - Entrenamiento: se realiza el ajuste de un modelo predefinido usando datos de ejemplo.
  - Validación: Se proponen y comparan diferentes modelos (diferente naturaleza, complejidad, variables utilizadas, etc.) para ver cuál es el más adecuado para el problema tras el correspondiente entrenamiento.
  - Test: Se cuantifica el rendimiento del modelo elegido.
- Para cada proceso se especifica un conjunto de datos diferente. El caso ideal es que todos procedan de la misma distribución. Lo más común es realizar una partición de los datos disponibles en tres conjuntos disjuntos,.

Entrenamiento (ajuste del modelo elegido a los datos)

- Partimos de *m* datos de ejemplo: $(x^{(i)}, y^{(i)})$. Serán los puntos azules.
- Pretendemos que nuestro modelo (curva roja) se ajuste al modelo real del proceso (curva verde, que no conocemos). Quereos "entrenarle" para predecir valores de salida a partir de valores de entrada.
- Este conjunto de datos se denomina de "entrenamiento", porque sirven para ajustar o entrenar nuestro modelo (le estamos entrenando para reproducir valores de salida ante valores de entrada).

Siguiendo el ejemplo, en este paso se aplica un descenso de gradiente o la ecuación normal de regresión para ajustar el coeficiente del modelo a los datos.

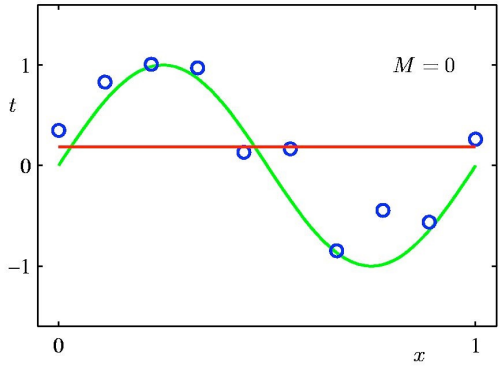


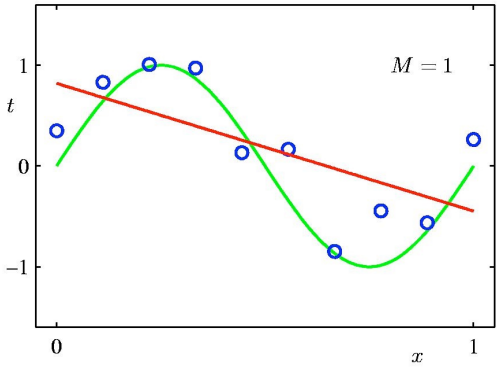
$$h(x,a) = h_a(x) = a_0 + a_1 x + \cdots + a_n x^n = \sum_{j=0} a_j x^j$$

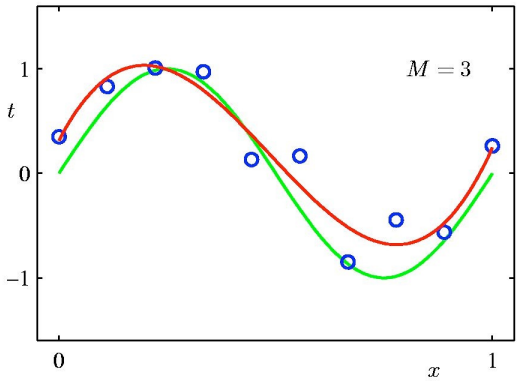
$$a = [a_0 \; a_1 \ldots a_n]$$

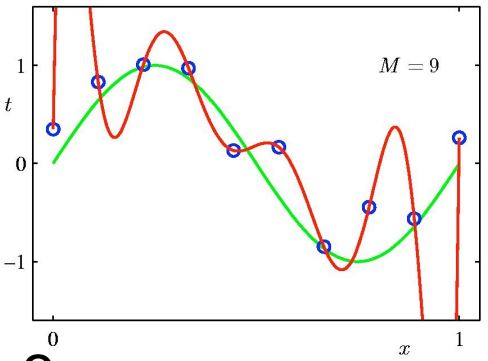
Entrenamiento: regresión

- Puede que dudemos del orden adecuado del modelo.
- Podemos decidir probar con diferente órdenes.





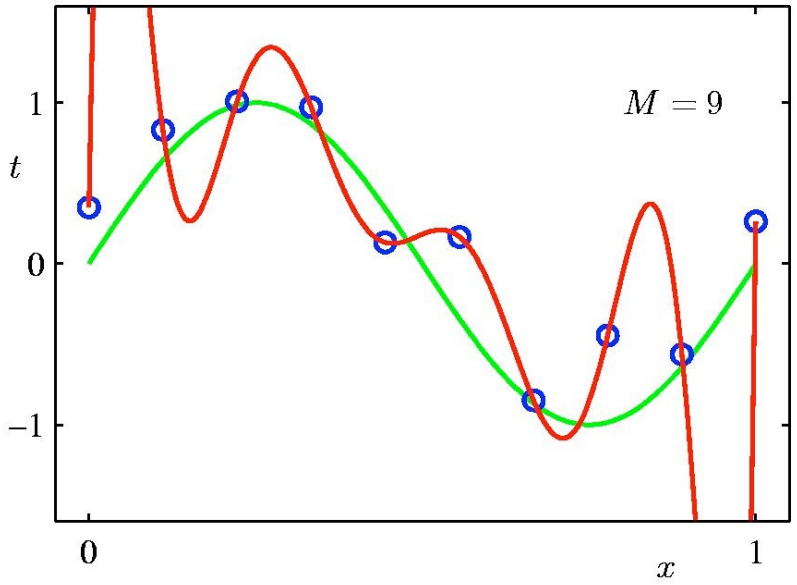




¿Con cuál nos quedamos?

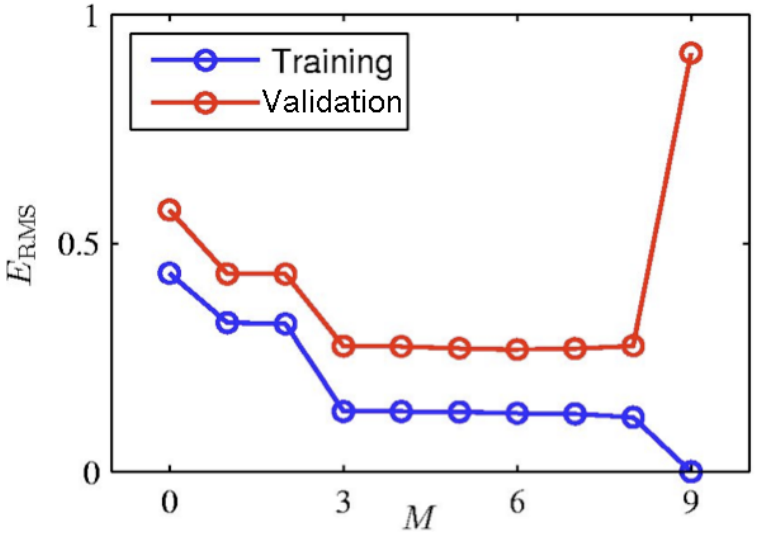
Sobreentrenamiento

- Nuestro modelo se ajusta excesivamente a los datos y no captura el comportamiento del proceso que los genera: no va a ser capaz de generalizar.



Validación (selección del modelo adecuado)

- Para decidir el orden recurrimos a un nuevo conjunto de datos de ejemplo denominado de validación.
- Tras ajustar el polinomio para cada orden, lo evaluamos con esos datos.
- Nos quedamos con el orden del polinomio que minimiza el “error de validación”.



Root-Mean-Square (RMS) Error:

$$E_{RMS} = \sqrt{2J(a)}$$

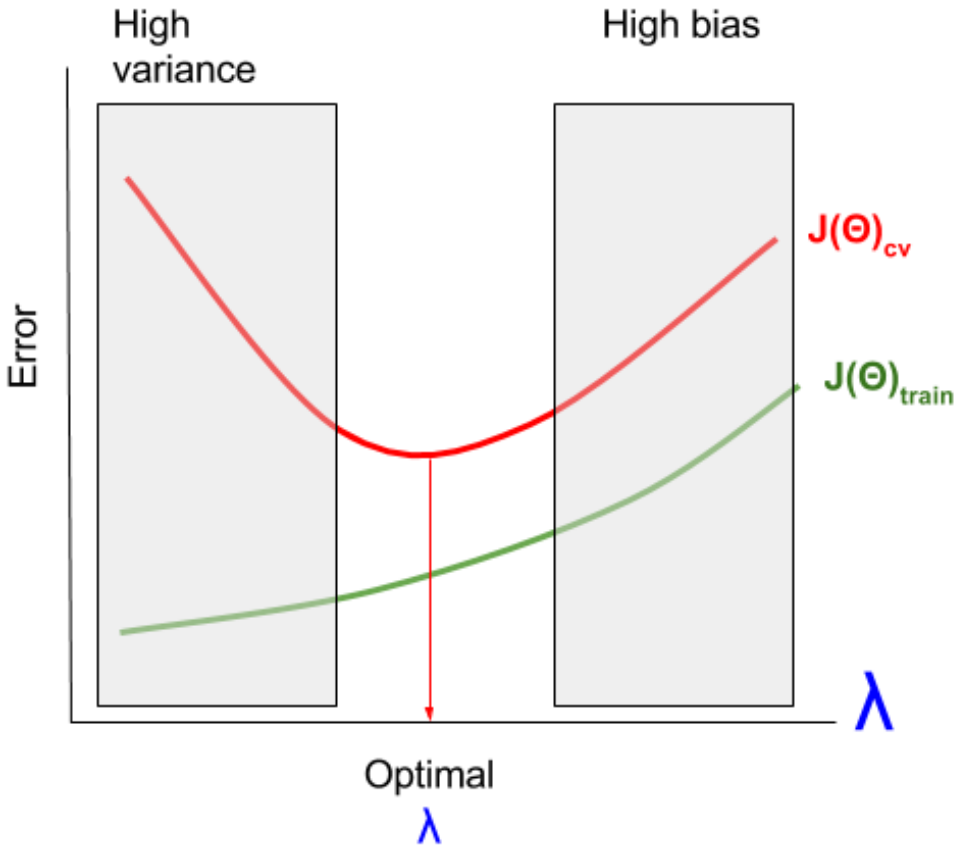
Regularización

- En diferentes modelos, y en especial en modelos de regresión, el sobreentrenamiento tiene como consecuencia un aumento considerable de la magnitud de los coeficientes.
- En algunos casos se puede penalizar la función de coste de manera proporcional a la magnitud de los parámetros.

$$J(a) = \frac{1}{2m}\sum_{i=1}^{m}(h_a(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} a_j^2$$

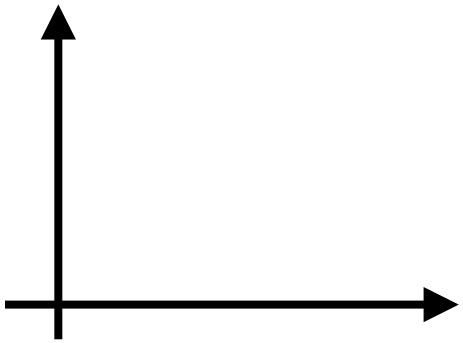
- Se penaliza así su crecimiento reduciendo la capacidad del modelo de sobreentrenar.
- Otros modelos de aprendizaje usan en ocasiones procedimientos de regularización similares.

Regularización



¿Por qué la regularización previene el sobreentrenamiento?

- En un caso como el de la regresión, la regularización ejercerá cierta presión que hará que ciertos coeficientes se anulen, lo cual supone de facto reducir el orden del polinomio.
- En cualquier caso, unos coeficientes bajos hacen que el comportamiento del polinomio sea prácticamente lineal.
  - Sea $y = a_0 + a_1 x + a_2 x^2 \ldots + a_n x^n$,



Test (prueba final del modelo)

- Finalmente, el modelo elegido suele probarse con un nuevo conjunto de datos (datos de test) para dar una idea de su capacidad de generalización.
- En ocasiones disponemos de un único juego de datos. Una política razonable a priori es usar el 70% para entrenamiento, el 15% para validación y el otro 15% para test.

Proceso Iterativo

- El “trabajo” real se desarrollará en el proceso de validación, donde se definen y depuran modelos.
- Lo aconsejable es comenzar con un algoritmo sencillo (veremos que para un mismo problema hay generalmente varias opciones, p.e. la regresión no tiene por qué ser polinómica, ni siquiera lineal). A partir de ahí ir verificando con los datos de validación diferentes alternativas de mejora.
- Al comenzar una nueva aplicación intentar siempre definir un conjunto de datos y una métrica en un tiempo corto. Es mejor empezar con algo imperfecto que dedicar a esto un esfuerzo elevado cuando aún no conocemos a qué nos enfrentamos. Si se trata de un problema maduro, habría que dedicar ya un esfuerzo importante en estas cuestiones para mejorar el estado del arte.
- En este proceso, disponer de una única métrica numérica ayuda considerablemente a la aceptación/descarte de propuestas.
- En ocasiones, este procedimiento iterativo puede conducir a sobreentrenar el modelo en el conjunto de validación. Esto se detectaría si una vez definido el modelo adecuado comprobásemos que el error de test es muy elevado. Si esto sucede, sería recomendable renovar el conjunto de validación y posiblemente el de test.